

1. A daily sales pipeline ran successfully at 6:00 AM, but the dashboard now shows revenue 40% lower than yesterday. You only have 30 minutes before the business team uses the dashboard in a meeting.

Please answer in this format:

1. What would you check first, second, and third?
2. What evidence would help you decide whether the problem comes from the source data, the transformation logic, or the dashboard layer?
3. Would you allow the dashboard to stay live, temporarily disable it, or add a warning message? Explain briefly.

1. What would you check first, second, and third?

- **First: data completeness and freshness.** Partial or late loads are the most common cause of a sudden, large drop, so I would check this first. I would compare today's row count and total revenue in the raw layer with yesterday's and with the same weekday last week. I would also check the latest order_timestamp and ingested_at values, because if they are much earlier than usual, the extraction probably ran before the source system finished its daily load. Finally, I would confirm that every expected partition or source file arrived, such as each region, store, and sales channel.
- **Second: where the revenue is lost.** I would reconcile the same revenue total across each layer, from raw to staging to mart to dashboard. The first layer where the value drops is where the problem is located. I would also break revenue down by key dimensions such as region, channel, and order status, and compare them with yesterday. A drop concentrated in one segment usually points to a missing source or a failed join, while a drop spread evenly across all segments suggests a filter, date range, or timezone issue.
- **Third: recent changes and business context.** I would review recent code deployments to the transformation models, upstream schema changes such as renamed columns or new status values, and dashboard configuration changes such as default filters or cache. At the same time, I would consider whether the drop could be genuine, for

example because of a public holiday, the end of a promotion, or a normally low-sales weekday.

2. What evidence would help you decide whether the problem comes from the source data, the transformation logic, or the dashboard layer?

- My main approach is to reconcile the same revenue figure across every layer. The first layer where the number drops tells me where the problem is, so I can isolate the cause by elimination rather than by guessing.
- **Source data issue.** The source is at fault if the raw layer already shows revenue or row counts about 40% lower than expected. This is usually supported by missing partitions or files for certain regions or channels, a latest record timestamp that is earlier than normal, or unexpected nulls and schema changes in the raw data. If the source system itself reports the same low figure, the problem is upstream, or the drop may be genuine.
- **Transformation logic issue.** The transformation layer is at fault if raw totals are normal but the mart totals are lower. Comparing row counts before and after each model shows exactly where records are lost. Common causes include an INNER JOIN to a dimension table that is missing new product or store IDs, an incremental model whose watermark skipped records, a changed filter condition, a missing exchange rate that produces null revenue, or a timezone change that shifted the daily cut-off.
- **Dashboard layer issue.** The dashboard is at fault if querying the mart directly with SQL returns the expected revenue while the dashboard shows a lower number. In that case I would check for incorrect default filters or date range, stale cached results, a timezone mismatch in the BI tool, a wrong aggregation such as AVG instead of SUM, or row-level security hiding part of the data.

3. Would you allow the dashboard to stay live, temporarily disable it, or add a warning message?

I would **keep the dashboard live with a clear warning message** and notify the meeting owner directly before the meeting.

Keeping it live without a warning is too risky, because the team could make decisions based on figures I already suspect are wrong. Disabling it entirely would remove access to other metrics that are likely still correct and would reduce trust in the dashboard. A warning such as *"Today's revenue figures are under investigation and may be incomplete. Please refer to yesterday's figures for decision-making"* keeps the dashboard usable while making the risk visible.

What happens next depends on what the investigation finds:

- **Confirmed error that cannot be fixed in time:** keep the warning or restore the last validated snapshot.
- **Partial load that can be fixed:** backfill, verify the totals, and remove the warning.
- **The drop is genuine:** remove the warning and briefly explain the cause to the business team.

After the incident, I would add automated data quality checks, including volume anomaly alerts, freshness checks, and row count reconciliation between layers. That way a "successful" run means the data is correct, not just that the pipeline finished.

2. You are designing a simple analytics data model for an online store. The business wants to answer these questions:

Please answer in this format:

1. What tables would you create?
2. What would be the grain of your main fact table?
3. Why would you include or not include an **order_items** table?
4. What problems would happen if product information were stored directly inside the **orders** table?

1. What tables would you create?

I would use a **star schema** with three fact tables and three dimension tables.

Dimension tables:

Table	Key columns
dim_customer	customer_key, customer_id, name, email, city, signup_date
dim_product	product_key, product_id, product_name, category, brand
dim_date	date_key, date, day_of_week, week, month, quarter, year

Fact tables:

Table`	Key columns
fact_order_items	order_id, product_key, customer_key, order_date_key, quantity, unit_price, discount_amount, line_revenue
fact_orders	order_id, customer_key, order_date_key, order_status, total_amount, item_count
fact_payments	payment_id, order_id, customer_key, payment_date_key, payment_method, amount, payment_status

2. What would be the grain of your main fact table?

The main fact table is fact_order_items . Its grain is **one row per product per order**, meaning one row per order line.

I chose this grain for three reasons:

- **It is the lowest level of detail the business needs.** Product-level questions like top-selling products cannot be answered from order-level data.
- **It can be aggregated upward.** Daily revenue, weekly revenue, and order totals can all be calculated from line-level data, but line-level detail cannot be recovered from an order-level table.
- **It keeps additive measures clean.** Quantity and line revenue can be summed safely across any dimension without double counting.

3. Why would you include or not include an `order_items` table?

I would include an **`order_items`** table, even though it is not listed as a business object. Orders and products have a many-to-many relationship, since one order can contain many products and one product can appear in many orders. An order-line table is the standard way to model this, and without it the question about top-selling products each week cannot be answered. It also stores facts that belong to each sale, namely quantity, discount, and the unit price at the time of purchase. Keeping the price on the order line means historical revenue stays correct even when the current product price changes. I would only skip this table if every order contained exactly one product, which is unrealistic for an online store.

4. What problems would happen if product information were stored directly inside the orders table?

The main problem is double counting. An order with several products would need one row per product, which repeats order-level values such as the order total. Summing those rows inflates revenue and order counts. The alternative, repeating columns like `product_1` and `product_2`, limits how many products an order can hold and makes queries very difficult.

It also causes redundancy and update anomalies. Product name, category, and brand would be repeated in every order row, so a change to one product would require updating many rows. If any are missed, reports become inconsistent.

Finally, it mixes product attributes with transaction facts. Product descriptions belong in `dim_product`, while quantity and price paid belong on the order line. Keeping them separate makes the model easier to query and maintain, and allows product changes to be tracked over time as a Slowly Changing Dimension if needed.

3. You are given a small CSV file containing online store order data. Each row represents one product item in an order.

1. Load the CSV file using Python.
2. Remove exact duplicate rows.
3. Remove rows where **quantity** or **unit_price** is missing.
4. Create a new column called **line_revenue = quantity * unit_price**.
5. Compute:
 - total revenue by day
 - total number of orders per customer
 - payment success rate (paid / total orders)

6. Print the final results clearly.

Code:

```
import pandas as pd

# 1. Load the CSV file

df = pd.read_csv("orders_sample.csv")

# 2. Remove exact duplicate rows (e.g. order 1007 is repeated twice,
identically)

df = df.drop_duplicates()

# 3. Remove rows where quantity or unit_price is missing (can't compute
revenue without them)

df = df.dropna(subset=["quantity", "unit_price"])

# 4. Create line_revenue = quantity * unit_price

df["line_revenue"] = df["quantity"] * df["unit_price"]

# 5a. Total revenue by day

revenue_by_day = df.groupby("order_date")["line_revenue"].sum().round(2)

# 5b. Total number of orders per customer

# (an order can have multiple product rows, so count distinct order_id,
not rows)
```

```
orders_per_customer = df.groupby("customer_id")["order_id"].nunique()

# 5c. Payment success rate (paid orders / total orders)

# payment_status is an order-level attribute, so dedupe to one row per
order first

orders = df.drop_duplicates(subset="order_id")

payment_success_rate = (orders["payment_status"] == "paid").mean()

# 6. Print the final results

print("=== Total revenue by day ===")

print(revenue_by_day.to_string())

print("\n=== Total orders per customer ===")

print(orders_per_customer.to_string())

print(f"\n=== Payment success rate ===")

print(f"{payment_success_rate:.1%} ({(orders['payment_status'] ==
'paid').sum()} of {len(orders)} orders paid)")
```

Result:

```
=== Total revenue by day ===
order_date
2026-04-01      77.00
2026-04-02      15.50
2026-04-03     154.00
2026-04-04     328.00
2026-04-05     133.97
2026-04-06     153.25
2026-04-07     265.50
```

```
=== Total orders per customer ===
customer_id
C001      3
C002      4
C003      2
C004      1
C005      1
C006      1
C007      1
C008      1
C009      1
C010      1
C011      1
C012      1
```

```
=== Payment success rate ===
66.7% (12 of 18 orders paid)
```


4. Using the data model you designed in Question 2 (customers, products, orders, order_items, payments), write SQL queries to answer the following:

- 1. Total revenue by day.**
- 2. Top 3 best-selling products each week (by quantity sold).**
- 3. Payment success rate (paid orders / total orders).**

Notes:

Standard SQL syntax is fine (e.g., PostgreSQL). You don't need to run or test it.

Briefly explain your logic in 1–2 sentences per query.

Feel free to state any assumptions about the table structure.

-- Data model:

- dim_customer(customer_key, customer_id, name, email, city, signup_date)
- dim_product(product_key, product_id, product_name, category, brand)
- dim_date(date_key, date, day_of_week, week, month, quarter, year)
- fact_order_items(order_id, product_key, customer_key, order_date_key, quantity, unit_price, discount_amount, line_revenue)
- fact_orders(order_id, customer_key, order_date_key, order_status, total_amount, item_count)
- fact_payments(payment_id, order_id, customer_key, payment_date_key, payment_method, amount, payment_status)

1. Total revenue by day

Logic: sum line_revenue from fact_order_items grouped by the order's date

(via dim_date), keeping only orders that have at least one 'paid' payment attempt.

```
SELECT
    d.date AS day,
    SUM(foi.line_revenue) AS total_revenue
FROM fact_order_items foi
```

```

JOIN dim_date d ON d.date_key = foi.order_date_key

WHERE EXISTS (

    SELECT 1 FROM fact_payments fp

    WHERE fp.order_id = foi.order_id AND fp.payment_status = 'paid'

)

GROUP BY d.date

ORDER BY day;

```

2. Top 3 best-selling products each week (by quantity sold)

Logic: aggregate quantity sold per product per ISO week from fact_order_items, then use RANK() to number products within each week by quantity descending, keeping rank <= 3.

```

WITH weekly_sales AS (

    SELECT

        date_trunc('week', d.date)::date AS week_start,

        foi.product_key,

        SUM(foi.quantity) AS total_qty

    FROM fact_order_items foi

    JOIN dim_date d ON d.date_key = foi.order_date_key

    GROUP BY 1, foi.product_key

),

ranked AS (

    SELECT

        week_start,

```

```

        product_key,

        total_qty,

        RANK() OVER (PARTITION BY week_start ORDER BY total_qty DESC) AS
rnk

    FROM weekly_sales

)

SELECT

    r.week_start,

    p.product_id,

    p.product_name,

    r.total_qty,

    r.rnk

FROM ranked r

JOIN dim_product p ON p.product_key = r.product_key

WHERE r.rnk <= 3

ORDER BY r.week_start, r.rnk;

```

3. Payment success rate (paid orders / total orders)

Logic: fact_payments is attempt-level, so collapse to one row per order that indicates whether it was EVER paid, then divide paid orders by all orders in fact_orders.

```

WITH order_payment_status AS (

    SELECT

        fo.order_id,

```

```
        BOOL_OR(fp.payment_status = 'paid') AS was_paid

    FROM fact_orders fo

    LEFT JOIN fact_payments fp ON fp.order_id = fo.order_id

    GROUP BY fo.order_id
)

SELECT

    COUNT(*) FILTER (WHERE was_paid)::numeric / COUNT(*) AS
payment_success_rate

FROM order_payment_status;
```